

CS 747: Programming Assignment 1

Akash Sansugu Palaniswami
21D171001

February 17, 2025

1 Task 1: Implementation of Sampling Algorithms

1.1 UCB

Upper Confidence Bound algorithm works by calculating a value for each arm of the bandit which predicts the upper bound of its probability. This is done using the following parameters:

- Total pulls of the bandit (t)
- Total pulls of the respective arm (u_a^t)
- Predicted probability of the arm using previous pulls ($\hat{p}_a^t$)

$$\text{ucb}_a^t = \hat{p}_a^t + \sqrt{\frac{2 \ln(t)}{u_a^t}}$$

```
class UCB(Algorithm):
    def __init__(self, num_arms, horizon):
        super().__init__(num_arms, horizon)
        # START EDITING HERE
        self.counts=np.zeros(num_arms)
        self.values=np.zeros(num_arms)
        self.ucb=np.zeros(num_arms)
        self.t=0
        # END EDITING HERE

def give_pull(self):
    # START EDITING HERE
    self.t += 1
    if (self.t==self.num_arms):
        return self.t-1
    sqrt_ln_t=math.sqrt(log(self.t))
    for i in range(self.num_arms):
        sqrt_u=math.sqrt(self.counts[i])
        self.ucb[i]=self.values[i]+ sqrt_ln_t/sqrt_u
    result = np.where(self.ucb == np.max(self.ucb))[0]
    if(len(result)==1):
        return result[0]
    else:
        a=np.random.randint(len(result))
        return result[a]
    # END EDITING HERE

def get_reward(self, arm_index, reward):
    # START EDITING HERE
    self.counts[arm_index]+=1
    n = self.counts[arm_index]
    value = self.values[arm_index]
    new_value = (value*(n-1)+reward)/n
    self.values[arm_index]=new_value
    # END EDITING HERE
```

(a) Variables Defined

(b) give-pull Function

(c) get_reward function

Figure 1: Code for UCB Algorithm

- Extra variables ucb and t have been defined. The variable ucb is an array which is used to store the upper confidence bound values of all the arms for each pull of the bandit. Variable t is just a scalar value used to keep track of the number of times the bandit is pulled
- In give_pull initially every arm is pulled once in round robin fashion to ensure u_a^t is never zero when calculating ucb. After ucb is calculated we take out the indices the maximum value in the ucb array. In the case of multiple indices having the same maximum value we randomly select one of them
- get_reward function updates u_a^t after each pull and calculates and updates the new $\hat{p}_a^t$ in the values array

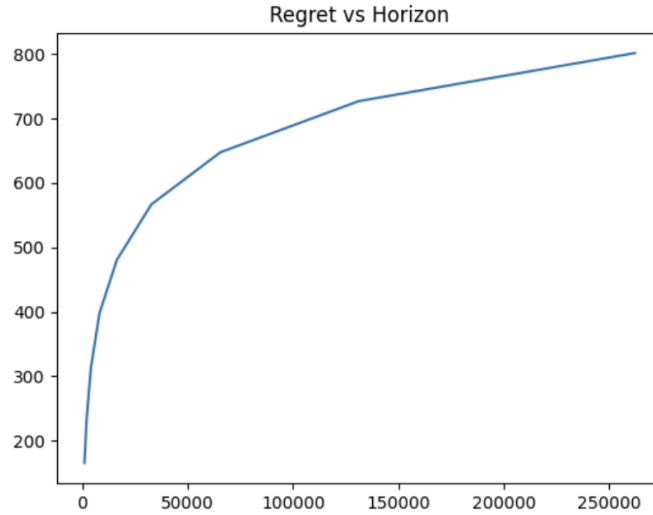


Figure 2: Regret plot for UCB Algorithm

For low horizon there is a steep increase in regret, but it stagnates at around 800 for horizons greater than 250000

1.2 KL-UCB

The method is similar to UCB, however the method to calculate the expected probability is slightly different. The parameters are also the same except that we now make use of KL divergence, and a new constant c , which is greater than or equal to 3. KL-UCB is defined as:

$$\text{ucb-kl } t_a^t = \max \{q \in [\hat{p}_a^t, 1] \text{ s.t. } u_a^t \text{KL}(\hat{p}_a^t, q) \leq \ln(t) + c \ln(\ln(t))\}$$

This can be simplified to:

$$q \in [\hat{p}_a^t, 1] \text{ to } KL(\hat{p}_a^t, q) = \frac{\ln(t) + c \ln(\ln(t))}{u_a^t}.$$

where q is the predicted probability

Functions defined for KL-UCB:

```
def KL(p,q):
    x=1e-7
    if(p==0):
        prob_p=x
    elif(p==1):
        prob_p=1-x
    else:
        prob_p=p
    if(q==0):
        prob_q=x
    elif(q==1):
        prob_q=1-x
    else:
        prob_q=q
    a=prob_p*log(prob_p/prob_q)
    b=(1-prob_p)*log((1-prob_p)/(1-prob_q))
    return a+b
```

(a) KL Divergence

```
def q_search1(p_hat,z,x,e=1e-6):
    l=p_hat
    h=z
    while (h-l>e):
        q=(l+h)/2
        k=KL(p_hat,q)
        error=x-k
        if (abs(error)<e):
            return q
        elif(error>0):
            l=q
        else:
            h=q
    return l/2+h/2
```

(b) Binary Search

```
def q_search_newton(p_hat,x,e=1e-4):
    q=p_hat/2+1/2
    k=KL(p_hat,q)
    error=x-k
    i=0
    while(abs(error)>e and i<50):
        i=i+1
        if q <= 0 or q >= 1:
            break
        d=p_hat/q-(1-p_hat)/(1-q)
        if (abs(d)<e):
            break
        q=q-error/(d)
        q = max(min(q, 1 - 1e-7), 1e-7)
        error=x-KL(p_hat,q)
    return q
```

(c) Newton Raphson

Figure 3: Functions Defined

- KL function is defined to calculate the KL divergence while taking into considerations if p or q is 0 or 1, because if that happens log values will be undefined. If this case occurs it is replace with an extrememly small value, or a value just less than 1.

$$KL(p, q) = p \ln \frac{p}{q} + (1 - p) \ln \frac{1 - p}{1 - q}$$

- In order to find the value of q, iterative methods have to be used, so i have implemented a binary search method as well as a newton raphson convergence method for this
- Similar to UCB, variables kl_ucb and t have been defined to keep track of the kl_ucb of each arm and the total pulls of the bandit. Additionally variable c has been kept at 3.
- In give_pull, once again round robin is implemented initially then kl_ucb is calculated for each arm and arm with maximum value is taken
- get_reward function updates the count of the arm and the mean probability of the arm which was pulled

```

class KL_UCB(Algorithm):
    def __init__(self, num_arms, horizon):
        super().__init__(num_arms, horizon)
        # You can add any other variables you need here
        # START EDITING HERE
        self.counts=np.zeros(num_arms)
        self.values=np.zeros(num_arms)
        self.kl_uct=np.zeros(num_arms)
        self.c=3
        self.t=0
        # END EDITING HERE

```

(a) Variables Defined

```

def give_pull(self):
    # START EDITING HERE
    self.t +=1
    if (self.t==self.num_arms):
        return self.t-1
    x=(log(self.t)+self.c*log(log(self.t)))
    for i in range(self.num_arms):
        y=x/self.counts[i]
        p=self.values[i]
        self.kl_uct[i]=log_search_newton(p,y)
    result = np.where(self.kl_uct == np.max(self.kl_uct))[0]
    if(len(result)==1):
        return result[0]
    else:
        a=np.random.randint(len(result))
        return result[a]
    # END EDITING HERE

```

(b) give_pull Function

```

def get_reward(self, arm_index, reward):
    # START EDITING HERE
    self.counts[arm_index] += 1
    n = self.counts[arm_index]
    value = self.values[arm_index]
    new_value = (value*(n-1)+reward)/n
    self.values[arm_index]=new_value
    # END EDITING HERE

```

(c) get_reward Function

Figure 4: Code for KL-UCB Algorithm

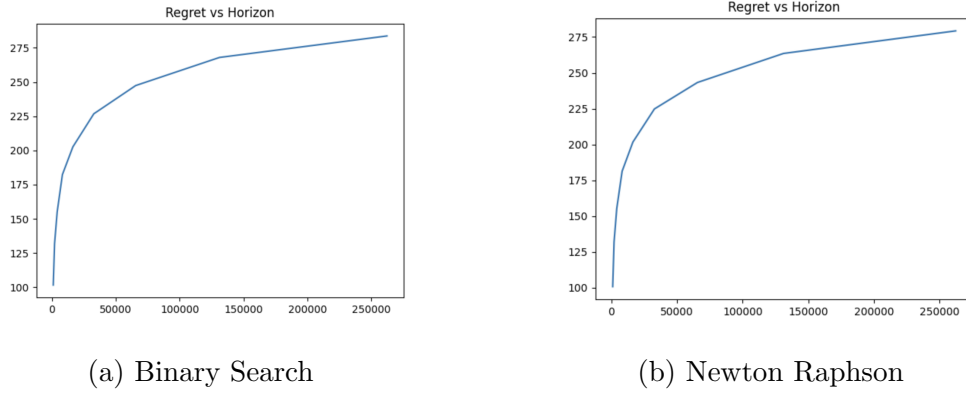


Figure 5: Regret Plots for KL-UCB

The regrets are more or less the same for Binary and Newton Raphson methods of searching, with them stagnating between 275-300, however the Newton Raphson method, took around 25 percent less time to run in simulator.py, when compared to binary search

1.3 Thompson Sampling

Thompson sampling works using the beta distribution. For each arm a beta distribution is created using the number of past successes and failures of that arm, and from there a random point is selected. Whichever arm contains the maximum value is then pulled

- Instead of a variable to count the number of pulls of an arm, two different variables are initialized to keep track of the number of successes and fails of each arm
- Numpy has a predefined function to for a beta distribution given the alpha and beta values which are number of successes +1 and number

```

class Thompson_Sampling(Algorithm):
    def __init__(self, num_arms, horizon):
        super().__init__(num_arms, horizon)
        # You can add any other variables you need here
        # START EDITING HERE
        self.s_counts=np.zeros(num_arms)
        self.f_counts=np.zeros(num_arms)
        self.thomson=np.zeros(num_arms)
        self.t=0
        # END EDITING HERE

```

(a) Variables Defined

```

def give_pull(self):
    # START EDITING HERE
    self.t += 1
    for i in range(self.num_arms):
        self.thomson[i]=mp.random.beta(self.s_counts[i]+1,self.f_counts[i]+1)
    result = np.where(self.thomson == np.max(self.thomson))[0]
    if len(result)==1:
        return result[0]
    else:
        amp=np.random.randint(len(result))
        return result[amp]
    # END EDITING HERE

```

(b) give_pull Function

```

def get_reward(self, arm_index, reward):
    # START EDITING HERE
    if reward==1:
        self.s_counts[arm_index] += 1
    elif reward==0:
        self.f_counts[arm_index] +=1
    # END EDITING HERE

```

(c) get_reward Function

Figure 6: Code for Thompson Sampling Algorithm

of fails +1 respectively. From this function a random value was picked from the distribution.

- get_reward function updated the number of successes or fails of the pulled arm based on the reward

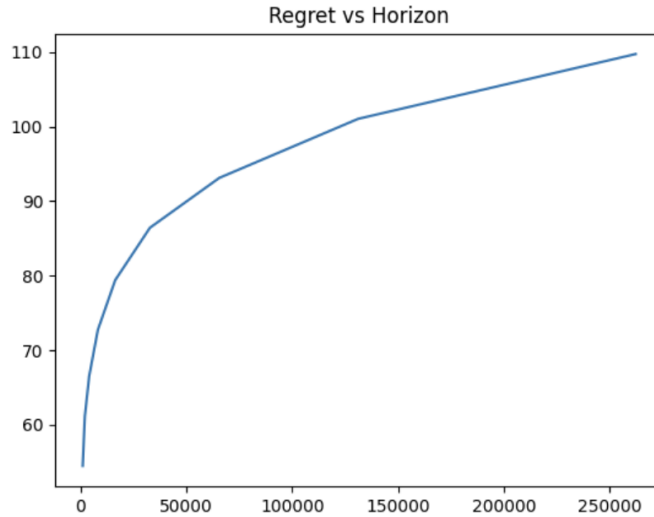


Figure 7: Regret plot for Thompson Sampling

We can clearly see that Thompson Sampling has the best results with regret already showing signs of stagnating at a regret of 110. KL-UCB seems to be the second best, with UCB having the highest regret values.

2 Task 2: Costly Bandit Setting

2.1 Approach

I decided to use Thompson Sampling for the base of my code as that showed us the best results by far in task 1. The main thing to do here, is to decide

upon the length of the query set that we will output. Assuming that the length of the query set is s . The query set will contain the indices of the s most maximum values which will be the sampling from the beta distribution of the different arms. Let the overall reward of giving a query set of length s be r . Cost of using query set of length s is $1/s$. The expected reward gotten from just giving out a query set and one arm being pulled would be the average expected probability of the arms in the query set. In our case the expected probability would be the random sampling of the beta distribution.

$$r = (\sum_{i=1}^s p_i - 1)/s$$

p_i represents probability of arm with index i in query set. So for every iteration of our bandit, we calculate value of r for every value of s from $[1, \text{number of arms}]$, and whichever value of s gives us maximum reward we use that for that iteration. We then select arms with the s most maximum values of random sampling of beta distribution and give those arm indices as query set.

2.2 Implementation Details

- Similar to Thompson's Sampling we define two arrays to keep count of number of successes and failures of each arm. We define an array s so that we can store the value of total reward for different values of s . Horizon is also defined and fixed, unlike in task 1
- In `give_query_set` function, initially through one for loop an estimated probability is assigned to each arm by a beta distribution. Then we iterate through the different values that s can take. For each iteration we take the s best arms, then have another for loop to sum all the probabilities and get the mean. Through this process the expected total reward is calculated for each value of s , then the best value is taken.
- `get_reward` just updates the number of successes and fails of the selected arm

```

def give_query_set(self):
    # START EDITING HERE
    for i in range(self.num_arms):
        self.values[i]=np.random.beta(self.s_counts[i]+1,self.f_counts[i]+1)
    for i in range(self.num_arms):
        indices=np.argmaxpartition(self.values,-(i+1))[-(i+1):]
        total=0
        for j in range(len(indices)):
            total=total+self.values[indices[j]]
        self.s[i]=(total-1)/(i+1)
    result = np.where(self.s == np.max(self.s))[0]
    if(len(result)==1):
        m=result[0]
    else:
        a=np.random.randint(len(result))
        m=result[a]
    return np.argmaxpartition(self.values,-(m+1))[-(m+1):]

    # END EDITING HERE

```

(a) give_query_set Function

```

class CostlySetBanditsAlgo(Algorithm):
    def __init__(self, num_arms, horizon):
        # You can add any other variables you need here
        self.num_arms = num_arms
        self.horizon = horizon
        # START EDITING HERE
        self.s_counts=np.zeros(num_arms)
        self.f_counts=np.zeros(num_arms)
        self.values=np.zeros(num_arms)
        self.s=np.zeros(num_arms)
        # END EDITING HERE

```

(b) Variables Defined

```

def get_reward(self, arm_index, reward):
    # START EDITING HERE
    if reward==1:
        self.s_counts[arm_index] += 1
    else:
        self.f_counts[arm_index] +=1
    #END EDITING HERE

```

(c) get_reward Function

Figure 8: Code for Task 2

2.3 Observations

Rewards increases linearly with the change in horizon, so it means the program gives consistent rewards for an increase in rewards.



Figure 9: Reward vs Horizon Plot

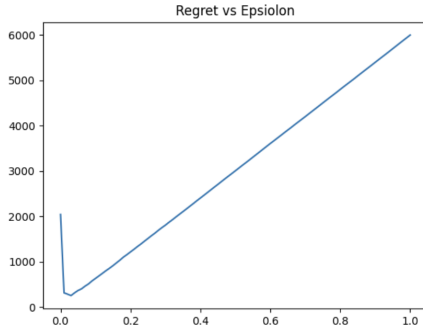
3 Task 3: Effect of Epsilon in Epsilon-Greedy Algorithm

3.1 Implementation

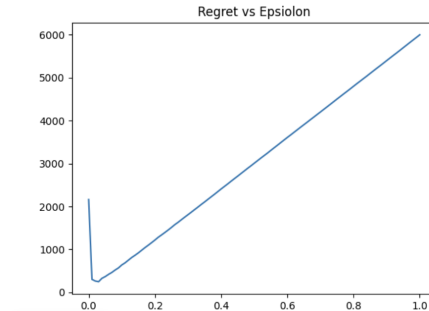
Most of the code is taken directly from task 1 and simulator.py other than a few things. Instead of keeping a fixed epsilon and iterating through different horizons, here we keep a fixed horizon of 30000 and iterate epsilon values from $[0,1]$ at a step of 0.01. The key edits in single_sim and simulate functions is just to input the value of epsilon. 2 different tiebreaker rules were implemented. First scenario was just a random selection if a tiebreak occurred. In the second scenario the arm with lowest amount of prior pulls was selected in case of a tiebreak. If multiple arms had the same minimum pulls, then random selection was made between 2 arms in the order of iteration.

3.2 Results

Both tiebreakers more or less give us the same results. The optimal epsilon is 0.03 with a regret of 247.912 for both of them.



(a) First Tiebreaker



(b) Second Tiebreaker

Figure 10: Regret vs Epsilon

3.3 Observations

- Although the optimal epsilon and regret is the same for both tiebreakers, we can see that the second tiebreaker has a slightly higher regret for very low values of epsilon
- An epsilon of 0.03 shows that very little exploration is required till we are able to find the best arm to pull
- The reason for the low value of epsilon could be that the horizon is pretty large when compared to the number of arms, so even with an epsilon of 0.03, it would mean that 900 pulls were used for exploration, so each arm would have been pulled on average around 45 times. That is more than enough to get a good idea about its probability. If the horizon was less, the required epsilon might increase
- Another reason that epsilon could be less is because none of the arms have a real probability that is close to each other. There is a minimum difference of 0.5. This could make it easy to figure out which arm is better. Maybe if the difference in probabilities was less, the optimal epsilon might be higher